

DevOps Interview Preparation Series (Final Terraform Session)

 by T Keerti

1) How do you manage Terraform state (location, safety, locking, backups)?

Answer:

- **Remote backend** (S3 + DynamoDB lock prevents local drift, enables team access, and provides **locking** to avoid concurrent writes.
- **Versioning & encryption** (SSE-KMS) on the bucket; **least-privilege IAM** for read/write; **bucket lifecycle** for backups.
- **Workflows**: `init` (migrates state to remote), `state pull` (inspect), `state mv/rm` (refactors), `import` (adopt existing).
- **Policy**: one state per **environment/workload**; don't share a single state across unrelated stacks.
- **Drift detection**: run `terraform plan` regularly in CI.

Command	Purpose	Example	Impact
<code>terraform init -migrate-state</code>	Migrate local to remote backend	Move state to S3/DynamoDB	Enables team collaboration
<code>terraform state pull</code>	Inspect state content	Debug state mismatches	Read-only
<code>terraform state mv</code>	Move resource in state	Rename or modularize	Non-destructive refactor
<code>terraform state rm</code>	Remove resource from state	Stop tracking resource	Resource remains in cloud
<code>terraform import</code>	Adopt existing infra	Import manual EC2	Adds to Terraform management

2) What happens if the state file is deleted? How do you recover?

- **Check remote versioning/backup** → restore latest.
- If no backup, **recreate state** by running `terraform import` for each live resource (script it), then `plan` to align config ↔ reality.
- Lock down access to prevent repeats; enable bucket versioning

3) Can Terraform fail if multiple runs happen in parallel?

Answer:

- With proper **state locking**, one apply will wait and the other proceeds; without locking you risk corruption.
- Even with locking, avoid parallel applies for the **same workspace**

4) Remote backends: which and how to configure?

Answer:

- **AWS**: S3 backend with DynamoDB for locks.

```
terraform {
  backend "s3" {
    bucket  = "tf-state-prod"
    key     = "network/terraform.tfstate"
    region  = "ap-south-1"
    dynamodb_table = "tf-locks"
    encrypt = true
  }
}
```

5) How do you handle sensitive data (passwords, keys) in Terraform?

Answer:

- **Never** hard-code secrets. Use:
 - Cloud secrets managers (AWS Secrets Manager/SSM, Azure Key Vault, GCP SM) via **data sources** or injected as env at deploy time.
 - `terraform.tfvars` in CI secrets store; mark as **sensitive** in variable definitions.
 - `-var/TF_VAR_` only from CI secret context.
- Prevent leakage: `sensitive = true` on outputs; scrub logs; use `tfint/checkov` policies

6) Reusable modules: how do you design them?

Answer:

- **Structure**: `variables.tf`, `outputs.tf`, `main.tf`, `README.md`, example usage.
- **Contracts**: typed variables with validation blocks; minimal required inputs; sensible locals.
- **Versioning**: pin module versions (`?ref=v1.3.0`) and providers (`required_providers`).
- **Composition**: child modules per concern (network, compute, db) rather than one giant root.

7) Handling dependencies between resources

Answer:

- Prefer **implicit** deps via interpolations (resource A references B's attribute).
- For non-obvious order, use `depends_on`.
- For replacements, use `lifecycle { create_before_destroy = true }` and occasionally `ignore_changes` for drift-tolerant fields.

8) Integrating Terraform into CI/CD

Answer:

- Stages: `fmt` → `validate` → `tfint` → **security** (Checkov) → `plan` (save artifact) → **manual approval** → `apply`.
- Use **workspaces** or separate folders/states per env; inject var files from CI.

9) `plan` vs `apply` — why always plan first?

Answer:

- `plan` computes intended changes (and drift) safely; output is reviewable and can be **signed off**.
- `apply` executes that plan. In CI, persist the **exact** plan file and feed it into `apply` to avoid TOCTOU changes.

10) Provisioners: when to use?

Answer:

- **Avoid** when possible (prefer cloud-init, user-data, Packer, config management).
- If needed: `local-exec` for side-effects (notify, register), `remote-exec` for bootstrapping with `timeouts/on_failure = continue`.

11) Testing Terraform code

Answer:

- **Static**: `terraform validate`, `tfint`, `checkov`.

12) Useful outputs & when to use them

Answer:

- Expose connection info (LB DNS, VPC IDs) for **downstream pipelines**.
- Mark as sensitive where needed; consume outputs across stacks via remote state data source or CI artifacts.

13) Multiple accounts / regions strategy

Answer:

- **Provider aliases** + **assume_role** per account; loop accounts with `for_each` at the **root module**.
- Separate **states per account+env**; guard with policies.

14) If someone changed infra manually (outside Terraform), how do you detect & fix?

Answer:

- `plan` will show drift. Decide to **import** (keep) or **reapply** (overwrite).
- Prevent future drift via **permissions** (no console writes), OPA/Sentinel policies, and "break-glass" SOPs.

15) Data sources: what and why?

Answer:

- **Read-only** lookups (e.g., latest AMI, existing VPC, secret value).
- They **don't** create resources—just feed attributes into your config.

16) Creating multiple instances without repeating code

Answer:

- Use `count` for homogeneous lists; `for_each` for maps/sets with stable keys.
- Combine with **dynamic blocks** to build nested arguments.

17) Destroy or change just one resource without impacting the rest

Answer:

- Targeted operations:
 - Destroy: `terraform destroy -target aws_instance.web[0]`
 - Create/replace: `terraform apply -target module.db` or `-replace=aws_lb.app` (newer flag).
Use sparingly—can violate graph; prefer normal `apply` when possible.

18) Upgrading Terraform versions & provider changes

Answer:

- Read the **upgrade guides**; bump versions in `required_version` and `required_providers`; run `init -upgrade`.
- Use **moved** blocks when renaming resources to avoid recreation; run `plan` in a throwaway workspace first.
- Provider breaking changes can require argument/attribute updates.

19) What if an apply fails halfway?

Answer:

- Terraform is **convergent**. Fix the cause, then `re-apply`; TF reconciles remaining steps.
- If partials created unusable combos, use `-replace` or `taint` (legacy) to force recreation; sometimes manual cleanup is needed

21) Managing multiple environments (dev/stage/prod)

Answer:

- Prefer **separate states** (and often separate repos) per env.
- Distinct backends, buckets, and IAM; promote via CI using **versioned modules**.
- Workspaces are okay for **light** env diffs; for real isolation use separate folders and states.

22) Frequently used Terraform commands (and why)

Answer:

`fmt`, `init`, `validate`, `plan`, `apply`, `destroy`, `graph`, `output`, providers lock, workspace list/select, `import`, `state mv/rm`, `taint/-replace`.

21. What is Terraform and how is it different from ARM/CloudFormation templates?

Terraform is an **Infrastructure as Code (IaC)** tool that lets you provision infrastructure declaratively across **multiple clouds**.

- **Terraform** → Multi-cloud (AWS, Azure, GCP, etc.), HCL syntax, state management.
- **ARM/CloudFormation** → Single-cloud (Azure/AWS), JSON/YAML format, tied to vendor.

22. What are Terraform modules?

Modules are **reusable collections of resources** grouped for easier management (like VPC, EC2, or AKS modules).

- Encourages DRY (Don't Repeat Yourself).
- Standard structure: `main.tf`, `variables.tf`, `outputs.tf`.
✅ Example:

```
module "network" {
  source = "../modules/vpc"
  cidr_block = "10.0.0.0/16"
}
```

23. What are data blocks and when are they used?

Data blocks **fetch existing resources** instead of creating them.

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["ubuntu/images/*"]
  }
}
```

24. How do you manage Terraform state files?

- Store remotely in **S3/DynamoDB** (AWS) or **Azure Blob Storage**.
- Enable **locking** to avoid parallel changes.
- Enable **versioning** for rollback.
✅ Example backend:

```
terraform {
  backend "s3" {
    bucket  = "tf-state-prod"
    key     = "network/terraform.tfstate"
    region  = "ap-south-1"
    dynamodb_table = "tf-lock"
  }
}
```

25. What happens if the Terraform state file gets deleted?

- Recover from **bucket versioning** backup.
- Or re-import resources using `terraform import`.
- Then verify using `terraform plan`.

26. Why does the state file get locked?
Locking ensures **no concurrent write**.
Unlock using:

```
terraform force-unlock <lock-id>
```

27)How do you pass dynamic variables (VM size, name, region, etc.)?

- `terraform.tfvars` file
- CLI: `terraform apply -var="vm_size=Standard_B2s"`
- Environment variable: `TF_VAR_vm_size=Standard_B2s`

28)What are meta-arguments?
Arguments applied to all resources:

- `count`
- `for_each`
- `depends_on`
- `lifecycle { create_before_destroy, ignore_changes }`
- `provider`

29. How do you provision a Windows VM with pre-installed software?
Use `custom_data` or `provisioners`:

```
resource "azurerm_virtual_machine_extension" "install" {
  name                = "install-app"
  virtual_machine_id  = azurerm_windows_virtual_machine.vm.id
  publisher           = "Microsoft.Compute"
  type                = "CustomScriptExtension"
  settings = jsonencode({
    "commandToExecute" = "powershell.exe -ExecutionPolicy Unrestricted -File install.ps1"
  })
}
```

30)How do you roll back a failed Terraform deployment?

- Terraform doesn't support rollback directly — `re-run` with last good state or restore from versioned backend.
- Also consider:
 - Use plan artifact approval.
 - Apply in stages (`network` → `compute` → `DB`).

31)16. How do you manage secrets while provisioning a VM?

- Use **Azure Key Vault / AWS Secrets Manager**.
- Access via data sources:

```
data "azurerm_key_vault_secret" "dbpass" {
  name = "db-password"
  key_vault_id = data.azurerm_key_vault.example.id
}
```

- Pass them as variables (`sensitive = true`).
- Avoid plaintext in logs or `.tfvars`.

32)How do you manage sensitive outputs?
Mark them as:

```
output "db_password" {
  value     = data.azurerm_key_vault_secret.dbpass.value
  sensitive = true
}
```

33). What if Terraform plan shows no changes, but infra was modified manually?
That's **drift**.
Fix by:

- Running `terraform refresh` or `plan` to detect drift.
- Re-`import` or re-`apply` depending on desired state.
- Prevent using limited IAM access.

34)How do you create AKS or EKS using Terraform?
Use cloud-specific providers (`azurerm_aks_cluster` / `aws_eks_cluster`), then manage node pools, scaling, and networking via separate modules.

37) How do you ensure idempotency in Terraform?

- Never use random values without `random_id` resource.
- Avoid provisioners for stateful changes.
- Keep consistent variable values and version locks.

38)How do you perform drift detection and validation in CI/CD?

- Run `terraform plan -detailed-exitcode` in nightly CI jobs.
- If exit code = 2 → infra drift detected → alert team.

1. What is Terraform?

Terraform is an **open-source Infrastructure as Code (IaC)** tool created by HashiCorp. It lets you **define, provision, and manage infrastructure** using a **declarative configuration language (HCL)**.

♦ **Key Idea:**

Instead of manually creating resources in AWS/Azure Console, you define them in `.tf` files and Terraform handles the creation, modification, and deletion automatically.

♦ **Real-World Example:**

```
resource "aws_instance" "web" {
  ami      = "ami-08d4ac5b634553e16"
  instance_type = "t2.micro"
}
```

2)How does Terraform work with AWS?

Terraform communicates with AWS services using the **AWS Provider plugin** and AWS **API calls**.

♦ **Workflow:**

1. Terraform reads your configuration.
2. It authenticates to AWS using access keys or IAM roles.
3. It compares your code's desired state with the real AWS environment (via AWS APIs).
4. It plans the changes and applies them.

♦ **Example Provider Block:**

```
provider "aws" {
  region = "ap-south-1"
  profile = "dev"
}
```

3. What is an AWS provider in Terraform?

A **provider** in Terraform is a plugin responsible for understanding API interactions for a specific platform (AWS, Azure, GCP, GitHub, etc.).

♦ **Example:**

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

4. How do you define resources in Terraform?

A **resource block** defines infrastructure components (EC2, S3, VNet, etc.) in HCL syntax inside `.tf` files.

♦ **Example:**

```
resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-devops-bucket"
  acl    = "private"
}
```

💡 Each resource has:

- **Type** (e.g., `aws_s3_bucket`)
- **Name** (logical reference)
- **Arguments** (properties)

5. What is a Terraform state file?

Terraform stores information about your infrastructure in a **state file** — `terraform.tfstate`. This state maps real resources (in AWS/Azure) to your Terraform configuration.

♦ **Example Content:**

```
{
  "resources": [
    {
      "type": "aws_instance",
      "name": "web",
      "instances": [{ "id": "i-023bd..." }]
    }
  ]
}
```

6. How can you initialize a Terraform project?

`terraform init`
Downloads provider plugins

- Initializes backend (e.g., S3/Azure Blob)
- Prepares for `plan` and `apply`

💡 Always re-run `terraform init` after adding new providers or modules.

What is Terraform?

Open-source IaC tool that lets you provision and manage infra as code using HCL.

How does Terraform work?

Creates a graph of resources, compares desired vs current state from state file, and executes API calls to reach the desired state.

Terraform vs CloudFormation

Terraform = multi-cloud, HCL, state file. CloudFormation = AWS-only, YAML/JSON, no external state.

Explain the Terraform workflow

init → plan → apply → destroy.

What is the state file and why important?

Tracks resource IDs and metadata → enables incremental changes and drift detection.

Local vs Remote State

Local = solo use; Remote = team collaboration (S3, GCS, AzureRM).

How do you enable state locking?

S3 backend with DynamoDB table (hash key LockID).

How to secure state files?

Encrypt S3 with KMS, restrict IAM policies, enable versioning, disable public access.

How do you manage secrets?

Use SSM/Secrets Manager/Vault data sources, mark outputs sensitive = true.

What is a provider?

Plugin responsible for API interaction (e.g., AWS, Azure).

What is a resource?

Definition of infra object like VPC, EC2, S3.

Difference between resource and data source?

Resource creates infra; data source reads existing infra.

How to create multiple instances?

Use count or for_each.

What is a module?

Reusable bundle of resources (main.tf, variables.tf, outputs.tf).

What are variables and outputs?

Inputs parametrize infra; outputs expose values after apply.

What is a workspace?

Separate state for different envs (dev/stage/prod).

What is a backend?

Defines where state is stored (local/S3/GCS/Azure).

Explain provisioners.

Run scripts on resources (local-exec/remote-exec).

How do you handle drift?

terraform plan detects changes; apply to reconcile.

Explain depends_on.

Adds explicit dependency edges when not implicit.

Explain create_before_destroy.

Ensures new resource built before old destroyed → zero downtime.

What is ignore_changes?

Ignore certain attributes to avoid perpetual diffs.

Explain terraform import.

Adopt existing resource into state.

How to recover lost state?

Restore from backend versioning or import live resources.

How to test Terraform code?

validate, tfint, checkov, and Terratest (Go).

CI/CD integration flow

fmt → validate → tfint → plan → manual approval → apply.

How to prevent parallel applies?

DynamoDB lock + CI pipeline queueing.

How to manage multi-account AWS?

Provider aliases + assume_role; one state per account/env.

What is terraform taint / -replace?

Force resource recreation.

When apply fails halfway?

Fix issue → re-apply; Terraform is idempotent.

Difference between count and for_each

count = list/index; for_each = map/keyed.

What is the Terraform registry?

Hub for pre-built modules (community & official).

Explain locals.

Simplify complex expressions inside code.

Explain data sources with example.

Fetch existing AMI/VPC → feed into new resources.

How to migrate backend?

terraform init -migrate-state.

How to manage Terraform versions?

required_version, required_providers, lock file .terraform.lock.hcl.

What is Terraform Cloud?

SaaS backend offering remote exec, workspaces, Sentinel policies.

Policy as Code in Terraform?

Sentinel/OPA enforces security/compliance (e.g., no public S3).

Zero Downtime Deployments?

create_before_destroy, blue-green ALB switch, Route 53 weighted records.

How to structure repos?

```
modules/  
  envs/dev/  
  envs/prod/
```

Each env = own backend & vars.

Common commands

init, fmt, validate, plan, apply, destroy, output, import, workspace.

How to use Terraform with Vault?

Data source vault_generic_secret to inject secrets.

Explain moved block.

Maps old addresses → new ones after refactor (prevents destroy/create).

Explain remote state data source.

Share outputs between stacks.

```
data "terraform_remote_state" "vpc" {  
  backend = "s3"  
  config = {  
    bucket = "tf-state"  
    key    = "vpc.tfstate"  
    region = "ap-south-1"  
  }  
}
```

Why is Terraform important for DevOps?

Bridges infra and automation; enables repeatable cloud deployments, auditability, and collaboration

1. What is CIDR in AWS/Terraform?

CIDR (Classless Inter-Domain Routing) defines the **IP address range** of your network or subnet. It is written as `<IP Address>/<Prefix Length>`, e.g. `10.0.0.0/16`.

In AWS (and Terraform), CIDR blocks are used to:

- Define **VPC** IP range (`cidr_block`).
- Define **subnet** IP range.
- Control **routing**, **security group ranges**, and **private/public** isolation.

2. How does CIDR notation work?

CIDR uses **prefix length** (the number after `/`) to represent the number of bits used for the network portion.

CIDR	Host Bits	Total IPs	Usable IPs	Common Use
<code>/8</code>	24 bits	16,777,216	~16M	Huge private corp network
<code>/16</code>	16 bits	65,536	65,534	Common VPC size
<code>/20</code>	12 bits	4,096	4,094	Medium subnet
<code>/24</code>	8 bits	256	254	Small subnet
<code>/28</code>	4 bits	16	14	Small management subnet
<code>/32</code>	0 bits	1	1	Single host IP

AWS reserves 5 IPs per subnet:

- 1st → Network address
- 2nd → VPC router
- 3rd → DNS
- 4th → Reserved
- Last → Broadcast address

So if a subnet has 16 IPs, only **11** are usable.

Example: Creating a VPC and Subnets with CIDR

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags = { Name = "prod-vpc" }
}

resource "aws_subnet" "public_a" {
  vpc_id = aws_vpc.main.id
  cidr_block = "10.0.1.0/24"
  availability_zone = "ap-south-1a"
  map_public_ip_on_launch = true
  tags = { Name = "public-a" }
}

resource "aws_subnet" "private_a" {
  vpc_id = aws_vpc.main.id
  cidr_block = "10.0.2.0/24"
  availability_zone = "ap-south-1a"
  tags = { Name = "private-a" }
}
```

Here:

- VPC has 65,536 IPs (`10.0.0.0 → 10.0.255.255`)
- Each subnet has 256 IPs (usable: 251)

4. How to divide a VPC into subnets?

Use **CIDR subnetting** to split your network logically.

Example:

VPC: `10.0.0.0/16`

Divide it into smaller subnets using Terraform's `cidrsubnet()` function.

```
variable "vpc_cidr" { default = "10.0.0.0/16" }

locals {
  public_subnet_cidrs = [cidrsubnet(var.vpc_cidr, 8, 0), cidrsubnet(var.vpc_cidr, 8, 1)]
  private_subnet_cidrs = [cidrsubnet(var.vpc_cidr, 8, 2), cidrsubnet(var.vpc_cidr, 8, 3)]
}

resource "aws_subnet" "public" {
  count = length(local.public_subnet_cidrs)
  vpc_id = aws_vpc.main.id
  cidr_block = local.public_subnet_cidrs[count.index]
  availability_zone = element(data.aws_availability_zones.available.names, count.index)
}

resource "aws_subnet" "private" {
  count = length(local.private_subnet_cidrs)
  vpc_id = aws_vpc.main.id
  cidr_block = local.private_subnet_cidrs[count.index]
  availability_zone = element(data.aws_availability_zones.available.names, count.index)
}
```

Explanation:

- `cidrsubnet(base, newbits, netnum)` splits CIDR dynamically.
- Adding 8 new bits splits `/16 → /24` subnets.
- `netnum` (`0,1,2,3...`) picks each subnet.

5. Real-world Subnet Planning Example

3-tier VPC design (commonly asked in Terraform interviews):

Tier	CIDR Range	Type	AZ	Purpose
<code>10.0.1.0/24</code>	Public	AZ-A	Frontend / Load Balancer	
<code>10.0.2.0/24</code>	Public	AZ-B	Frontend / Load Balancer	
<code>10.0.3.0/24</code>	Private	AZ-A	Application Servers	
<code>10.0.4.0/24</code>	Private	AZ-B	Application Servers	
<code>10.0.5.0/24</code>	DB	AZ-A	RDS Primary	
<code>10.0.6.0/24</code>	DB	AZ-B	RDS Standby	

6. Terraform Functions for Networking

Function	Purpose
<code>cidrsubnet()</code>	Divide a network into smaller subnets.
<code>cidrhost()</code>	Get specific IP from subnet.
<code>cidrnetmask()</code>	Get subnet mask of CIDR.

Examples:

```
cidrsubnet("10.0.0.0/16", 8, 0) → 10.0.0.0/24
cidrsubnet("10.0.0.0/16", 8, 1) → 10.0.1.0/24
cidrhost("10.0.0.0/24", 5)      → 10.0.0.5
cidrnetmask("10.0.0.0/24")     → 255.255.255.0
```

7.What is a CIDR block and why is it used in Terraform?

A CIDR block defines the range of IP addresses in a network. In Terraform, it is used in `cidr_block` attributes for resources like `aws_vpc`, `aws_subnet`, and routing.

8)What happens if two subnets overlap?

Terraform will throw an **AWS API error**:
`InvalidSubnet.Range: CIDR block overlaps with another subnet.`
You must ensure non-overlapping subnets.

9. What’s the difference between /16 and /24 in Terraform VPCs?

- `/16`: 65,536 total IPs → Good for large VPCs.
- `/24`: 256 IPs → Common for subnets.

AWS recommends starting with `/16` for flexibility.

10 How do you dynamically create multiple subnets from a single CIDR?

Use `cidrsubnet()` with `count` or `for_each` loops as shown earlier.

11) What is the ideal subnet size for public vs private?

- Public (Load Balancer, Bastion) → `/24`
- Private (App layer) → `/24`
- DB → `/28` or `/27` depending on instance count

12). What’s the maximum number of CIDR blocks per VPC?

You can have up to:

- **1 primary CIDR + up to 5 secondary CIDRs** per VPC (as of latest AWS update).
This can be used for IP expansion or overlapping migrations.

13) What’s the smallest and largest CIDR AWS supports?

- **Smallest subnet:** `/28` (16 IPs)
- **Largest VPC:** `/16` (65,536 IPs)

14)CIDR Troubleshooting in Terraform

Problem	Likely Cause	Fix
CIDR overlap error	Two subnets overlap	Adjust prefix using <code>cidrsubnet()</code>
Insufficient IPs	Too small CIDR	Increase subnet size
Peering fails	Overlapping CIDRs	Change one VPC's range
NAT/IGW route missing	Wrong route table association	Explicitly associate correct route tables
S3/VPC endpoint fails	Wrong subnet or route table	Verify endpoint and subnet mapping